FitSense AI – Personalized Fitness Recommendations Powered by AI

PROJECT TITLE: FitSense AI – Personalized Fitness Recommendations Powered by AI

1. **TEAM ID:** SWTID-2026-6934
2. **TEAM LEADER:** SAMUEL JOSHUA R & samrajan0210@gmail.com
3. **TEAM MEMBERS:**
    - ROOHITH S & roohith0529@gmail.com
    - SAMVEL M & mssamvel777@gmail.com
    - SATHISHKUMAR B & baskersathishbaskersathish@gmail.com

Introduction

FitSense AI is a secure, scalable, and AI-enabled RESTful backend created to make workout tracking easier and provide personalized fitness guidance. Developed with Node.js, Express.js, MongoDB, and Mongoose, the backend provides REST APIs for user authentication, workout management, workout searching, and AI-based fitness assistance.

The system follows the Model–View–Controller (MVC) pattern so that routing, business logic, database operations, middleware, and AI services remain organized into separate modules. JSON Web Tokens (JWT) are used to authenticate users, while bcrypt.js is used to hash passwords before they are stored in MongoDB. Protected endpoints use authentication middleware to verify JWT tokens before allowing access to private resources.

A major feature of FitSense AI is its integration with Google Gemini AI. The AI service uses information such as the user's age, fitness objective, and experience level to prepare personalized workout recommendations. It can also examine workout statistics such as the number of completed workouts, average workout duration, and calories burned to produce useful fitness insights.

All APIs return structured JSON responses, allowing the backend to be connected with web applications, mobile applications, or other client systems. Input validation, route protection, centralized error handling, and separate service modules improve reliability, security, and future extensibility.

Scenario-Based Case Study

Background

Rahul is a fitness enthusiast who works out regularly but maintains his exercise records manually in notebooks and spreadsheets. Even though he follows a consistent routine, organizing his workout history and measuring his progress is difficult. He also finds it challenging to decide whether his current exercises match his fitness objectives because he does not have personalized guidance.

Problem

- Workout information is scattered across manual records.
- Manual entry can result in incomplete or inaccurate workout history.
- Personal fitness data does not have secure user authentication.
- Finding older workout records is inconvenient.
- There is no automated personalized workout recommendation system.
- Users cannot easily obtain an overall analysis of their workout performance.
- Calculating workout duration and calorie information manually takes additional effort.

Solution

FitSense AI provides a centralized backend in which users can register, authenticate themselves, and maintain their workout records securely. Through RESTful APIs, authenticated users can create, retrieve, update, delete, and search workout information.

Google Gemini AI is connected through a dedicated service module. It uses user details and workout statistics to generate personalized workout recommendations and fitness insights. JWT authentication, password hashing, authorization middleware, schema validation, and centralized error handling are used to provide a secure and maintainable backend environment.

Usage

User Usage

A new user can register through the registration endpoint and log in using the registered email address and password. After successful authentication, the backend creates a JWT token. The token is supplied with protected requests so that the server can identify the authenticated user.

- Create workout records.
- View completed workout history.
- Retrieve a specific workout using its ID.
- Modify an existing workout.
- Remove unwanted workout records.
- Search workouts using name, category, or date.
- Request AI-generated workout recommendations.
- Request AI-generated fitness insights.

AI Usage

The AI functionality is implemented through a dedicated Google Gemini service. Users can provide the following information:

- Age
- Fitness Goal
- Experience Level

The recommendation service can return:

- Personalized workout plan
- Weekly exercise suggestions
- Recommendations suitable for beginner, intermediate, or advanced users
- Motivational guidance
- Exercise safety suggestions

The fitness-insight service can process workout statistics such as:

- Total Workouts
- Average Workout Duration
- Total Calories Burned

The resulting response can contain personalized observations and suggestions related to the user's workout activity.

Outcome

FitSense AI reduces the effort involved in manual fitness record keeping and adds AI-assisted guidance to the workout tracking process. JWT-based authentication helps ensure that personal workout information is accessible only to authorized users, while Gemini AI provides recommendations based on the information supplied by the user.

The modular MVC-based structure makes the backend easier to maintain and extend. Future versions can add nutrition tracking, BMI calculation, wearable-device connectivity, workout reminders, and fitness analytics dashboards without requiring a complete redesign of the backend.

1. Software Requirements

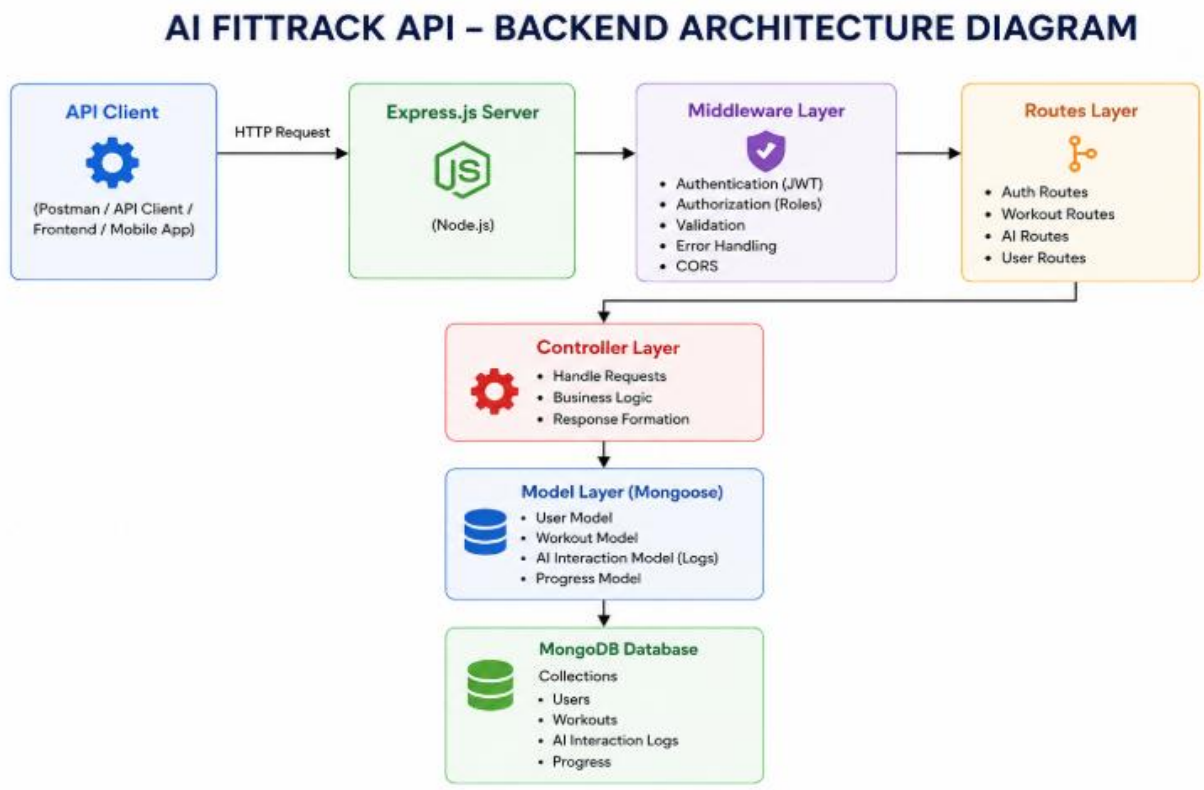
The following software is required to develop, run, and test the FitSense AI backend.

- Operating System: Windows 10/11, macOS, or Linux.
- Node.js (v16 or later): Provides the runtime environment for the backend.
- npm (v8 or later): Manages project dependencies and scripts.
- Express.js: Handles HTTP routing and REST API development.
- MongoDB: Stores user and workout information.
- Mongoose: Provides schema modeling and database interaction for MongoDB.
- Postman: Used to test and verify REST API requests and responses.
- Visual Studio Code or another suitable code editor.

2. Hardware Requirements

- Processor: Intel Core i5 (8th Generation or newer), AMD Ryzen 5, or equivalent.
- RAM: Minimum 8 GB; 16 GB is recommended when running MongoDB, the backend, and development tools together.
- Storage: At least 1 GB of free storage for the project and dependencies.

Epic 1: Project Architecture



FitSense AI uses a layered backend structure based on the MVC architecture. Routing, authentication, controllers, business services, database models, and AI communication are separated into individual modules. This separation makes the application easier to understand, test, maintain, and scale.

When a client sends an HTTP request, the Express server receives it and identifies the matching route. For protected resources, the authentication middleware first verifies the JWT token. The controller then validates the request and calls the required service or database model. Mongoose handles communication with MongoDB, while the Gemini service is responsible for AI-related requests. Finally, the controller sends a standardized JSON response to the client.

This structure gives each module a clear responsibility and reduces unnecessary dependency between components.

Overall Architecture Components

- Client Application (React / Mobile Application / Postman)
- Express Server
- API Routes
- Authentication Middleware
- Controllers
- Services
- Google Gemini AI Service
- MongoDB Database
- Mongoose Models

The components communicate through defined interfaces, allowing individual modules to be modified or extended with minimal impact on the rest of the system.

Components Definition

The FitSense AI backend is divided into functional layers, with each layer handling a specific responsibility. A request moves through these layers before reaching the database or AI service, which improves security and makes the application easier to manage.

1. Client Layer (Frontend/API Consumer)

The client layer is the entry point for users or external applications. It may be a React frontend, a mobile application, or Postman during testing. The client sends HTTP requests and receives JSON responses from the backend.

Responsibilities:

- Send API requests.
- Display workout information.
- Store and send the JWT after authentication.
- Call AI recommendation endpoints.
- Show success and error messages.

APIs accessed:

- Authentication APIs
- Workout APIs
- AI Recommendation APIs
- AI Fitness Insight APIs

2. Express Server Gateway

The Express server is the main entry point for incoming HTTP requests. It configures application middleware, reads JSON request bodies, enables CORS, registers routes, and passes requests to the appropriate modules.

- Start the HTTP server.
- Parse JSON request data.
- Enable Cross-Origin Resource Sharing (CORS).
- Register application routes.
- Connect authentication and other middleware.
- Handle application-level errors.

3. Authentication Middleware

Authentication middleware secures private endpoints by checking the JWT supplied in the Authorization header. A request is allowed to continue only when the token is valid.

- Read the Authorization header.
- Extract the JWT.
- Verify the token signature.
- Decode authenticated-user information.
- Attach user information to the request.
- Return an unauthorized response for invalid or missing tokens.

4. Route Layer

The route layer defines API endpoints and connects each HTTP request to its corresponding controller.

- Authentication Routes
- Register User
- Login User
- User Profile
- Workout Routes
- Add Workout
- Get All Workouts
- Search Workouts
- Get Workout by ID
- Update Workout
- Delete Workout
- AI Routes
- Workout Recommendation
- Fitness Insights

5. Controller Layer

Controllers coordinate the request-processing flow. They receive validated requests from the routes, call the necessary services or models, and send structured JSON responses.

Authentication Controller

- User Registration
- User Login
- User Profile Retrieval

Workout Controller

- Create Workout Records
- Retrieve Workout Records
- Update Workout Records
- Delete Workout Records
- Search Workout Records

AI Controller

- Prepare requests for Google Gemini
- Receive AI-generated output
- Return AI responses to the client

6. Service Layer

The service layer contains reusable application logic so that controllers remain focused on handling requests and responses.

Gemini AI Service

- Build AI prompts
- Send prompts to Google Gemini
- Receive Gemini responses
- Format recommendations and insights

JWT Service

- Generate JWT tokens
- Set token expiration
- Verify authenticated identity
- Password Service
- Hash passwords
- Compare passwords during login

7. Database Layer

MongoDB is responsible for persistent storage, while Mongoose provides the application with schema definitions, validation, and convenient database operations.

Collections

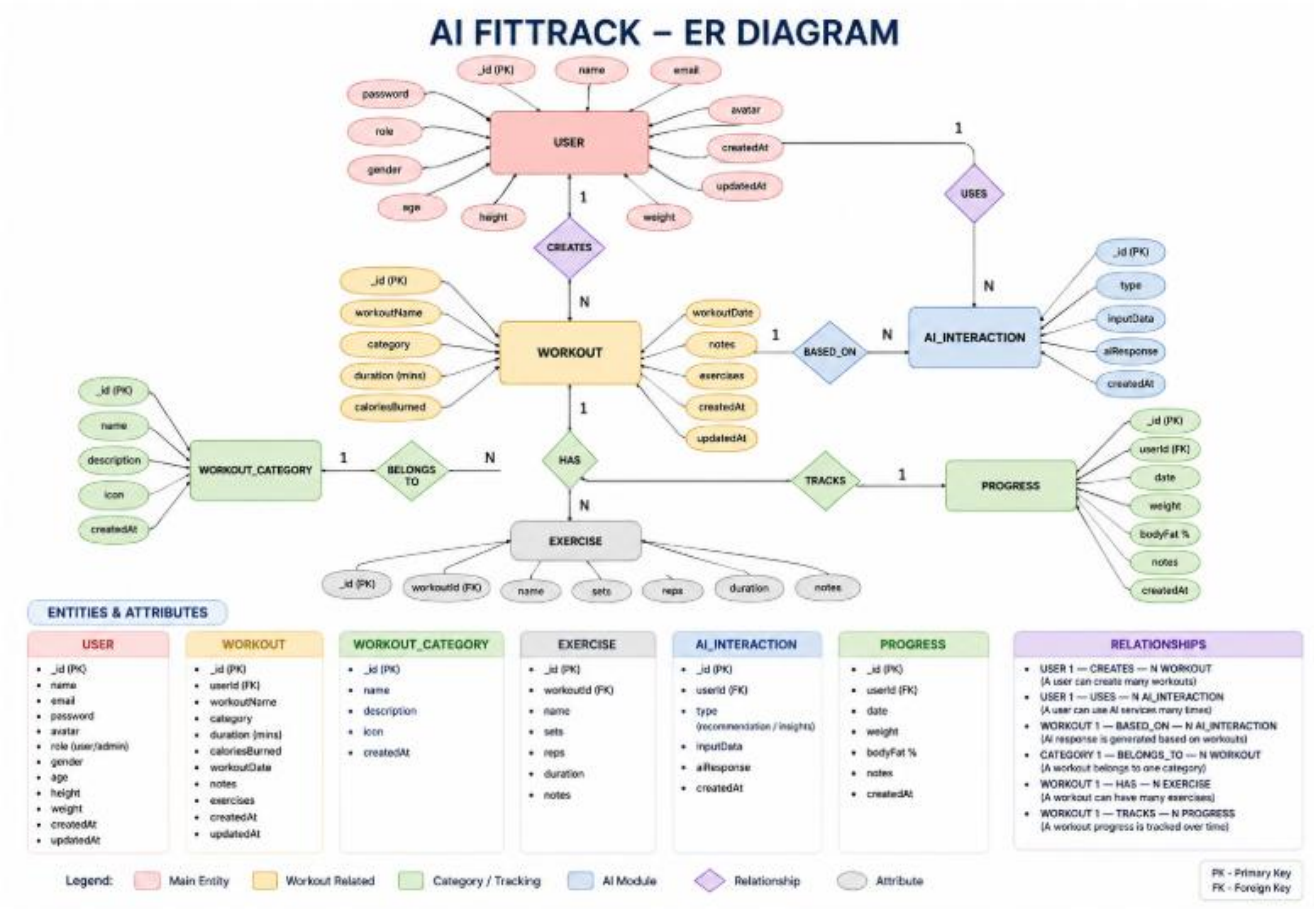
- Users Collection
- Workouts Collection

Advantages

- Flexible document structure
- Efficient data access
- Simple integration with Node.js
- Scalable storage model

Entity Relationship (ER) Diagram Description

The FitSense AI backend contains two main data entities: User and Workout. They are connected through a one-to-many relationship.



1. User Entity

The User entity contains account, authentication, and profile-related information.

Field	Type	Description
_id	ObjectId	Primary key
name	String	Name of the user
email	String	Unique email address
password	String	Hashed password
createdAt	Date	Time the record was created
updatedAt	Date	Time the record was last updated

Constraints

- Email address must be unique.
- Password is stored after bcrypt hashing.
- Name is required.
- A JWT is generated after successful login.

2. Workout Entity

The Workout entity stores the exercise records associated with an authenticated user.

Field	Type	Description
_id	ObjectId	Primary key
user	ObjectId	Reference to the User collection
workoutName	String	Name of the workout
category	String	Workout category
duration	Number	Workout duration in minutes
caloriesBurned	Number	Calories burned
workoutDate	Date	Date of the workout
createdAt	Date	Creation timestamp
updatedAt	Date	Last update timestamp

Relationship

User → Workout

Relationship Type: One-to-Many

One user can own multiple workout records, while every workout record is associated with one authenticated user. The connection is maintained through the user field in the Workout schema, which stores the ObjectId of the related user.

Example user records:

- Morning Running
- Evening Jogging
- Upper Body Strength
- Cycling Session

Key Features

FitSense AI combines secure workout management with AI-assisted fitness functionality.

1. Secure Authentication

- User Registration
- Secure Login
- Password Hashing
- JWT Token Generation
- Protected APIs

2. Workout Management

Authenticated users can perform complete CRUD operations on their workout records.

- Add Workout
- View Workout
- Update Workout
- Delete Workout

Every workout record is linked to its authenticated owner.

3. Workout Search

Workout history can be searched using:

- Workout Name
- Workout Category
- Workout Date

4. Google Gemini Integration

The backend sends structured user information to Google Gemini to obtain personalized workout guidance.

Input:

- Age
- Fitness Goal
- Experience Level

Output:

- Weekly Workout Plan
- Suggested Exercises
- Training Tips
- Safety Recommendations

5. AI Fitness Insights

Workout statistics can be sent to the AI service for analysis.

Input:

- Total Workouts
- Average Duration
- Calories Burned

Output:

- Performance Analysis
- Improvement Suggestions
- Motivational Advice
- Fitness Progress Summary

6. Centralized Error Handling

The backend provides meaningful JSON error responses for situations including:

- Invalid Credentials
- Missing Authentication Token
- Validation Errors
- Database Errors
- Internal Server Errors

7. RESTful API Design

The backend follows REST-oriented API conventions.

- GET
- POST
- PUT
- DELETE

API responses are provided in JSON format.

8. Modular Architecture

The project separates the following responsibilities:

- Models
- Controllers
- Routes
- Middleware
- Services
- Configuration

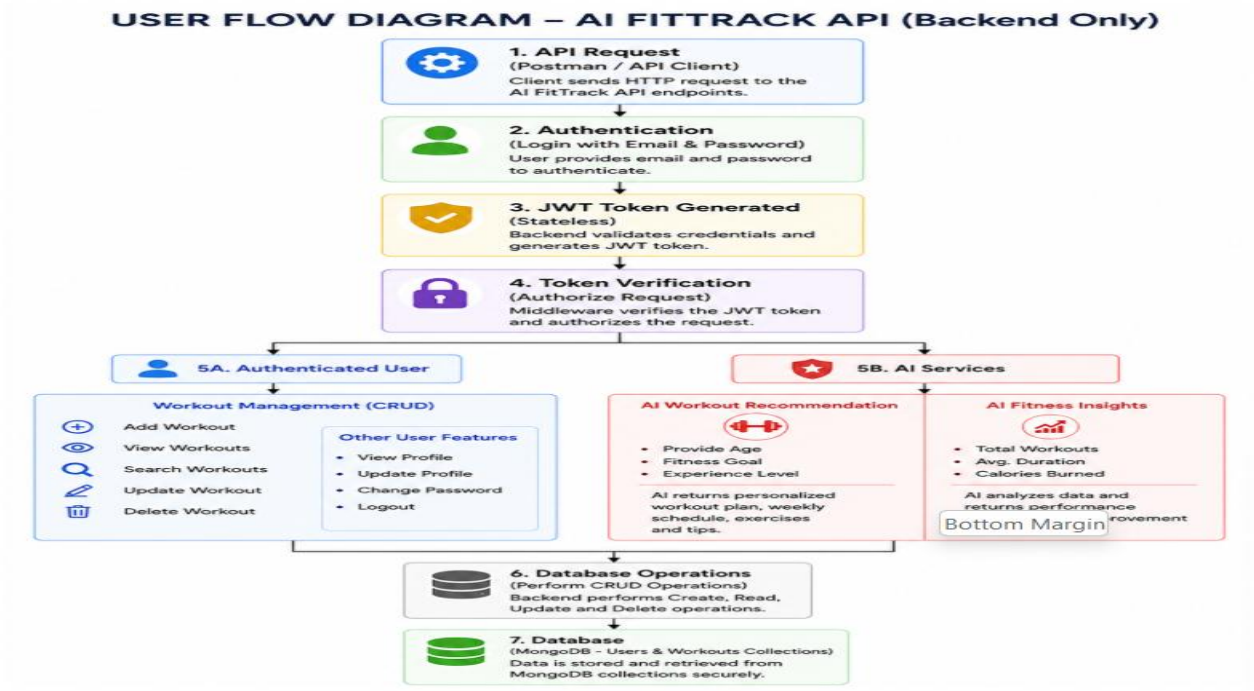
Roles and Responsibilities

Authenticated User

- Register an account.
- Log in securely.
- View profile information.
- Create workout records.
- View workout history.
- Search workout records.
- Update workout records.
- Delete workout records.
- Request AI workout recommendations.
- Request AI fitness insights.

Operation	Permission
Register	Yes
Login	Yes
View Profile	Yes
Add Workout	Yes
Get Workouts	Yes
Search Workouts	Yes
Update Workout	Yes
Delete Workout	Yes
AI Recommendation	Yes
Fitness Insights	Yes

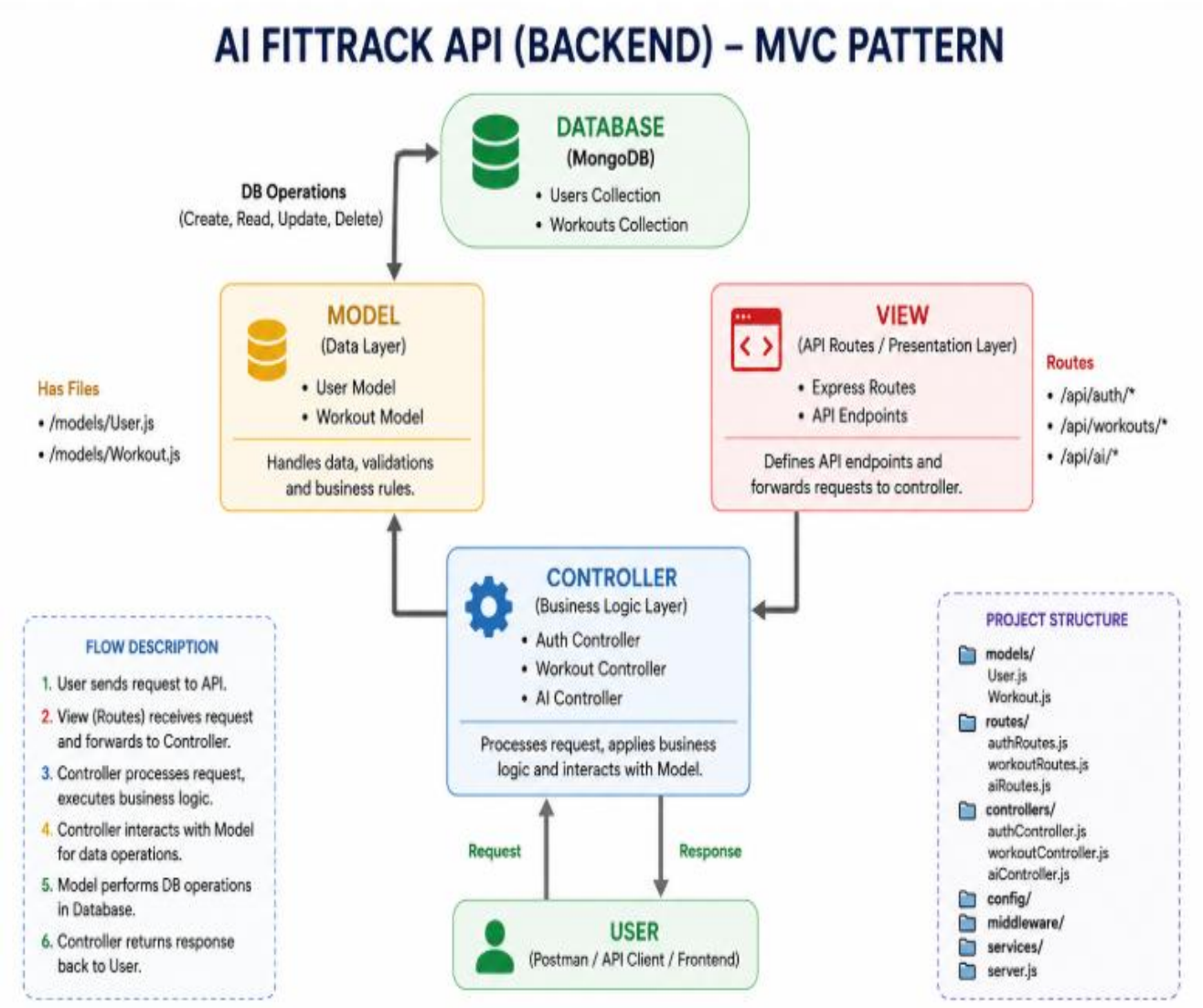
User Flow



- The user registers a new account.
- The user logs in with the registered credentials.
- The backend validates the credentials and issues a JWT token.
- The user sends the JWT token with protected API requests.
- The user creates or manages workout records.
- The user can search and review previous workouts.
- The user can request a personalized workout recommendation.
- The user can submit workout statistics to receive AI-generated fitness insights.

MVC Architecture Pattern

FitSense AI uses the Model–View–Controller pattern to separate data management, API interaction, and request-processing responsibilities. This arrangement improves organization, readability, maintainability, and scalability.



Model Layer (Data Layer)

The Model layer defines MongoDB document structures through Mongoose schemas and performs database-related operations.

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

View Layer (API Interface)

Because FitSense AI is a REST backend, it does not use traditional server-side UI templates. The View is represented by clients such as React applications, mobile applications, or Postman that send HTTP requests and consume JSON responses.

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

Controller Layer

The Controller layer connects the client request with the models and service modules. It validates incoming data, performs the required application logic, calls database or AI services, and returns the final JSON response.

- Request Validation
- Business Logic Execution
- Database Interaction
- AI Service Invocation
- Response Generation

Separating these layers reduces coupling, supports testing, improves code readability, and makes future feature additions easier.

Epic 2: Project Setup and Configuration

The FitSense AI backend is organized so that configuration, routes, controllers, database operations, authentication, middleware, and AI functionality are maintained separately. The project uses Node.js and Express.js for server development, MongoDB for data storage, and Google Gemini for AI-powered fitness guidance.

Creating Project Folder

Step 1: Create the Project Folder

- Create a new directory for the project:
- AI-FitSense-API

Step 2: Open the Project

- Open the project folder using Visual Studio Code.

Step 3: Create the Server Folder

- Inside the project directory, create:
- server
- The backend source code is maintained inside this folder.

Step 4: Initialize Node Project

- Open the terminal inside the server directory and run:
- npm init -y
- This creates package.json, which stores project information and dependency details.

Step 5: Install Required Packages

- npm install express mongoose dotenv bcryptjs jsonwebtoken cors
- Install Nodemon for development:
- npm install --save-dev nodemon

Environment Variables

Configuration values and sensitive credentials are kept in a .env file instead of being written directly into source files.

PORT=5000
MONGO_URI=mongodb://localhost:27017/aifitsense
JWT_SECRET=your_secret_key
GEMINI_API_KEY=your_google_gemini_api_key

Environment variables help keep credentials and deployment-specific configuration outside the application source code.

Epic 3: Backend Development

Backend development implements the REST APIs required for authentication, workout management, database access, and AI functionality. The implementation is divided into independent modules so each responsibility can be maintained separately.

- Server Configuration
- Database Configuration
- Authentication
- Workout Management
- AI Recommendation
- Middleware
- Error Handling

Epic 4: Database Configuration

MongoDB is the primary database used by FitSense AI for storing user accounts and workout records. Mongoose works as the ODM layer between the Node.js application and MongoDB, providing schemas, validation, relationships, middleware support, and simplified CRUD operations.

MongoDB Configuration

MongoDB stores data as BSON documents inside collections rather than using traditional relational tables. FitSense AI uses separate collections for users and workouts.

Database Collections

- Users Collection
- Workouts Collection

Installing Mongoose

- npm install mongoose

Mongoose provides schema definitions, validation rules, model methods, and database connectivity for the application.

MongoDB Connection



```
const mongoose = require('mongoose');

const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Error connecting to MongoDB: ${error.message}`);
    process.exit(1);
  }
};

module.exports = connectDB;
```

The database connection is maintained in config/db.js. The module connects the application to MongoDB when the server starts.

- Establish the MongoDB connection.
- Handle connection errors.
- Report the connection status.
- Export the database connection for application use.

Advantages:

- Centralized database configuration
- Reusable connection module
- Simpler connection management
- Clearer database error handling

Mongoose Schemas

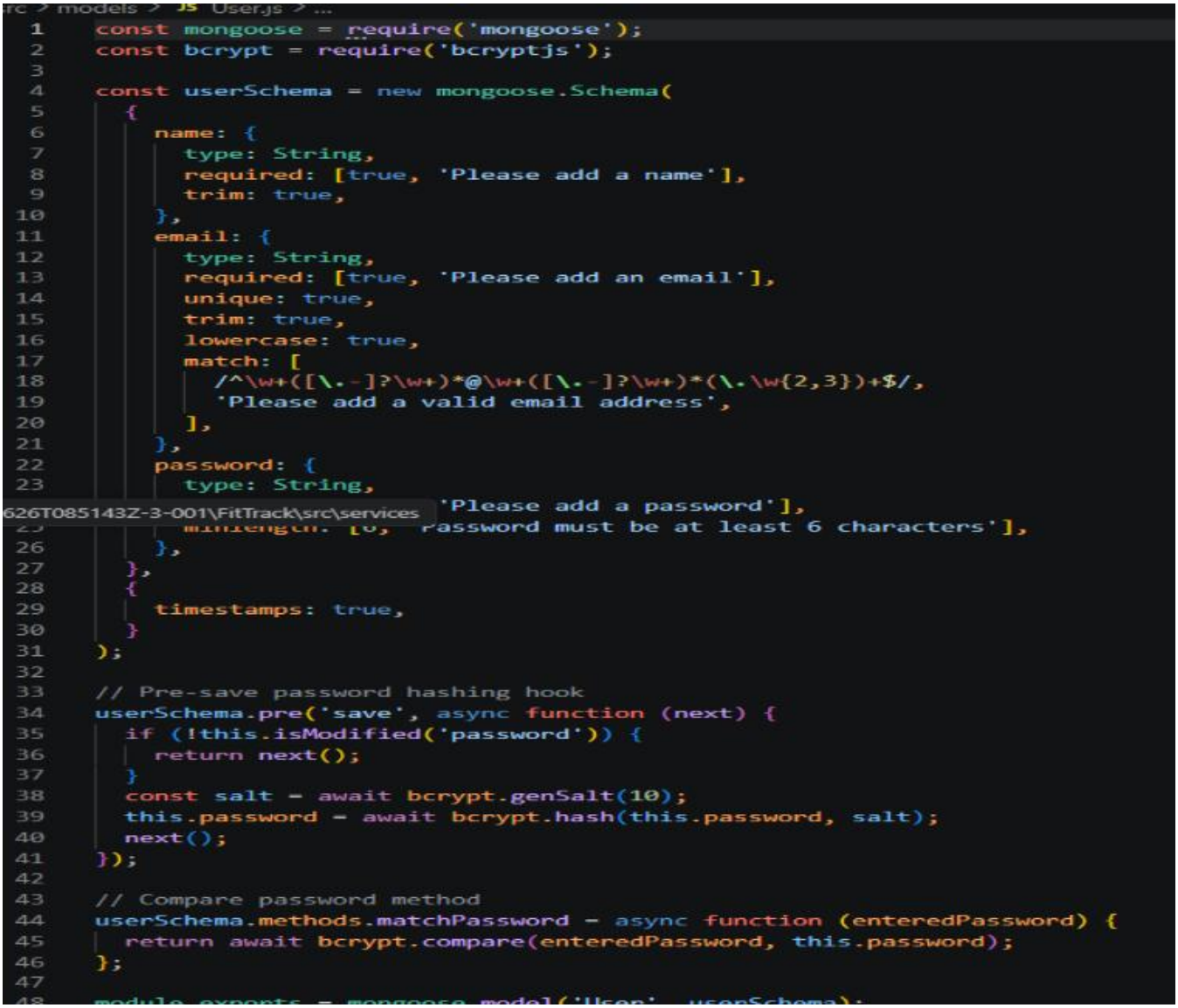
Two primary Mongoose schemas are used:

- User Schema
- Workout Schema

These schemas define the structure, validation rules, and relationship between stored documents.

1. User Schema (models/User.js)

The User schema stores account and authentication information.




```
const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema(
  {
    name: {
      type: String,
      required: [true, 'Please add a name'],
      trim: true,
    },
    email: {
      type: String,
      required: [true, 'Please add an email'],
      unique: true,
      trim: true,
      lowercase: true,
      match: [
        /^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/,
        'Please add a valid email address',
      ],
    },
    password: {
      type: String,
      'Please add a password'],
      minlength: [6, 'Password must be at least 6 characters'],
    },
  },
  {
    timestamps: true,
  }
);

// Pre-save password hashing hook
userSchema.pre('save', async function (next) {
  if (!this.isModified('password')) {
    return next();
  }
  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

// Compare password method
userSchema.methods.matchPassword = async function (enteredPassword) {
  return await bcrypt.compare(enteredPassword, this.password);
};

module.exports = mongoose.model('User', userSchema);
```

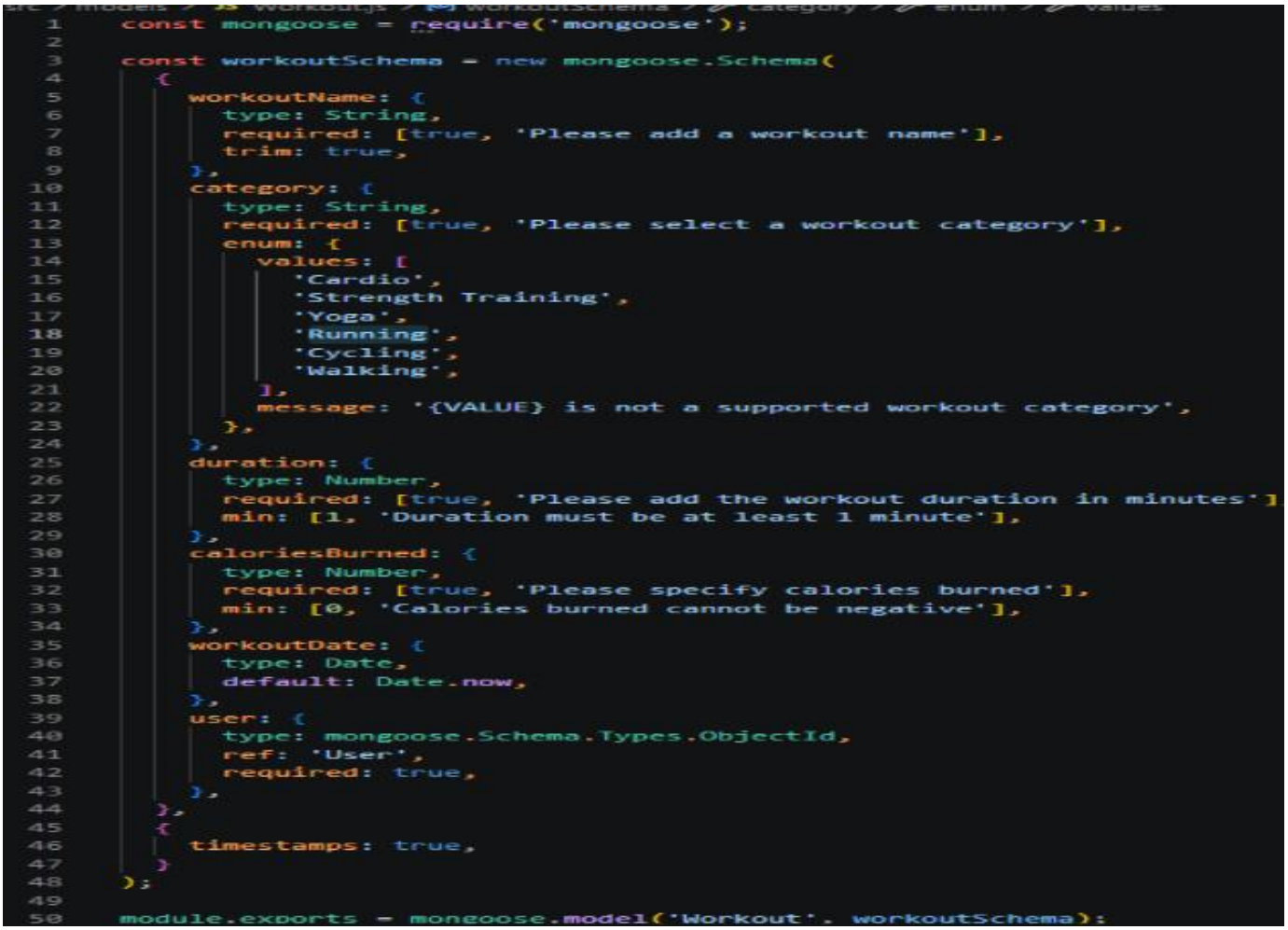
Validation Rules

- Name must be provided.
- Email must be unique.
- Password is required.
- Password is hashed using bcrypt.js before storage.
- Automatic timestamps are enabled.

User Schema Features

- Email uniqueness validation
- Password hashing
- Automatic timestamps
- Authentication support
- JWT-based login integration

2. Workout Schema (models/Workout.js)



```
const mongoose = require('mongoose');

const workoutSchema = new mongoose.Schema(
  {
    workoutName: {
      type: String,
      required: [true, 'Please add a workout name'],
      trim: true,
    },
    category: {
      type: String,
      required: [true, 'Please select a workout category'],
      enum: {
        values: [
          'Cardio',
          'Strength Training',
          'Yoga',
          'Running',
          'Cycling',
          'Walking',
        ],
        message: '{VALUE} is not a supported workout category',
      },
    },
    duration: {
      type: Number,
      required: [true, 'Please add the workout duration in minutes'],
      min: [1, 'Duration must be at least 1 minute'],
    },
    caloriesBurned: {
      type: Number,
      required: [true, 'Please specify calories burned'],
      min: [0, 'Calories burned cannot be negative'],
    },
    workoutDate: {
      type: Date,
      default: Date.now,
    },
    user: {
      type: mongoose.Schema.Types.ObjectId,
      ref: 'User',
      required: true,
    },
  },
  {
    timestamps: true,
  }
);

module.exports = mongoose.model('Workout', workoutSchema);
```

The Workout schema contains the exercise information created by authenticated users.

Validation Rules

- Workout name is required.
- Category is required.
- Duration must be greater than zero.
- Calories burned must be zero or greater.
- Workout date is required.
- Each workout is associated with one authenticated user.

Workout Schema Features

- Schema validation
- Automatic timestamps
- User reference
- Search support
- CRUD operation support

Epic 5: Project Execution

Install Postman or use Thunder Client as a VS Code extension.

Start the backend server using:

npm start

API Testing

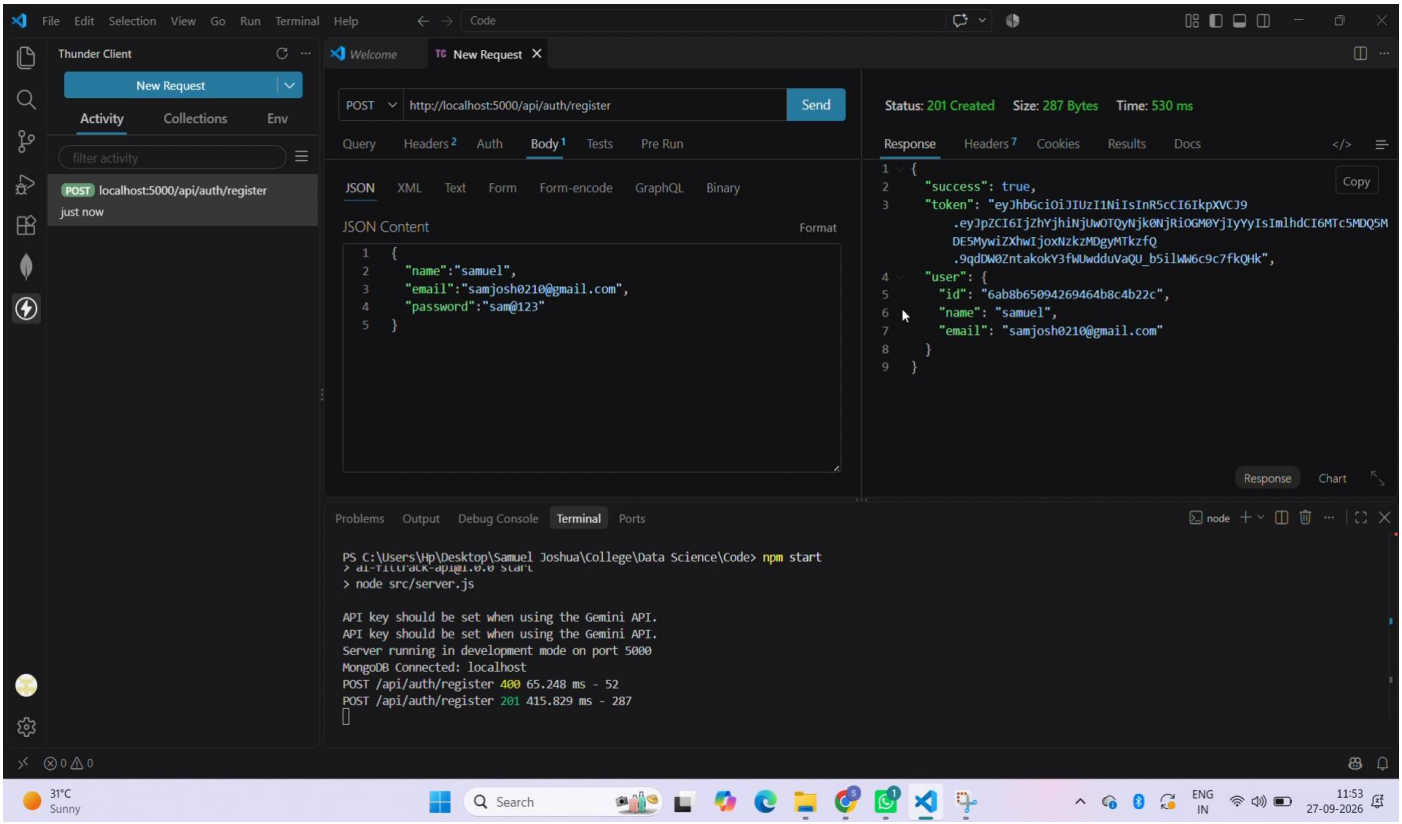
API testing is used to confirm that each endpoint accepts the expected request, performs the intended operation, and returns the correct response. Postman can be used to test authentication, workout operations, authorization, database interactions, and AI endpoints.

Testing is carried out in the following order:

- Authentication APIs
- Workout APIs
- AI APIs

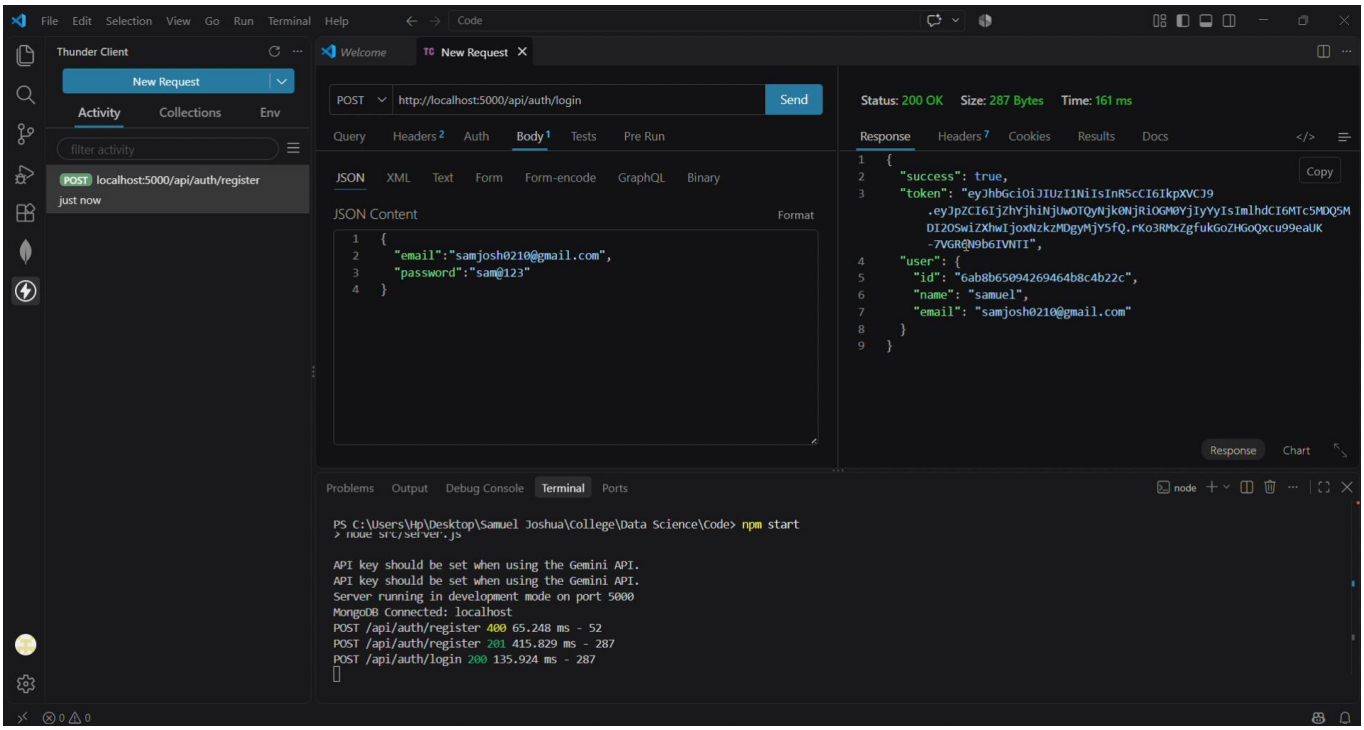
Authentication API Testing

Register User



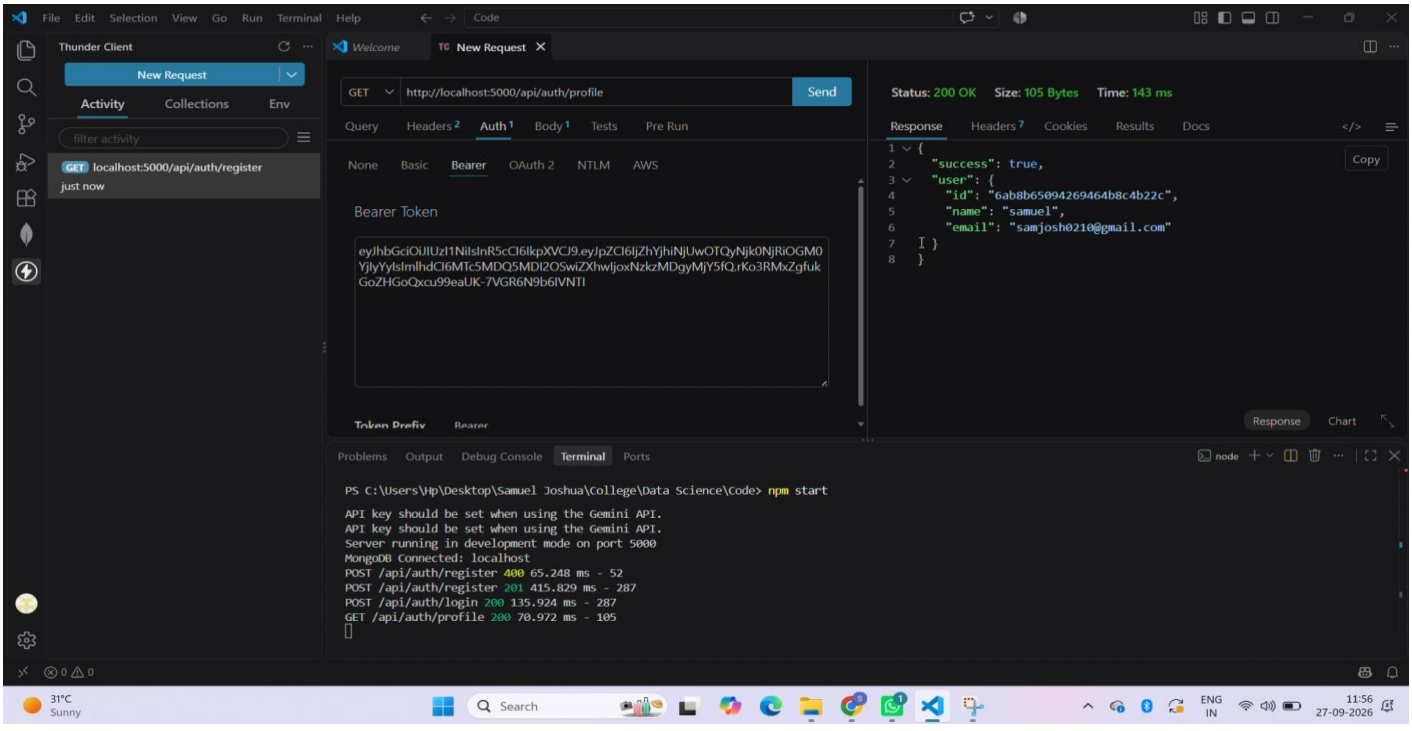
Test the registration endpoint by submitting the required user details and verifying the returned success or validation response.

Login User



Test login using the registered email and password. A successful request returns a JWT token for accessing protected endpoints.

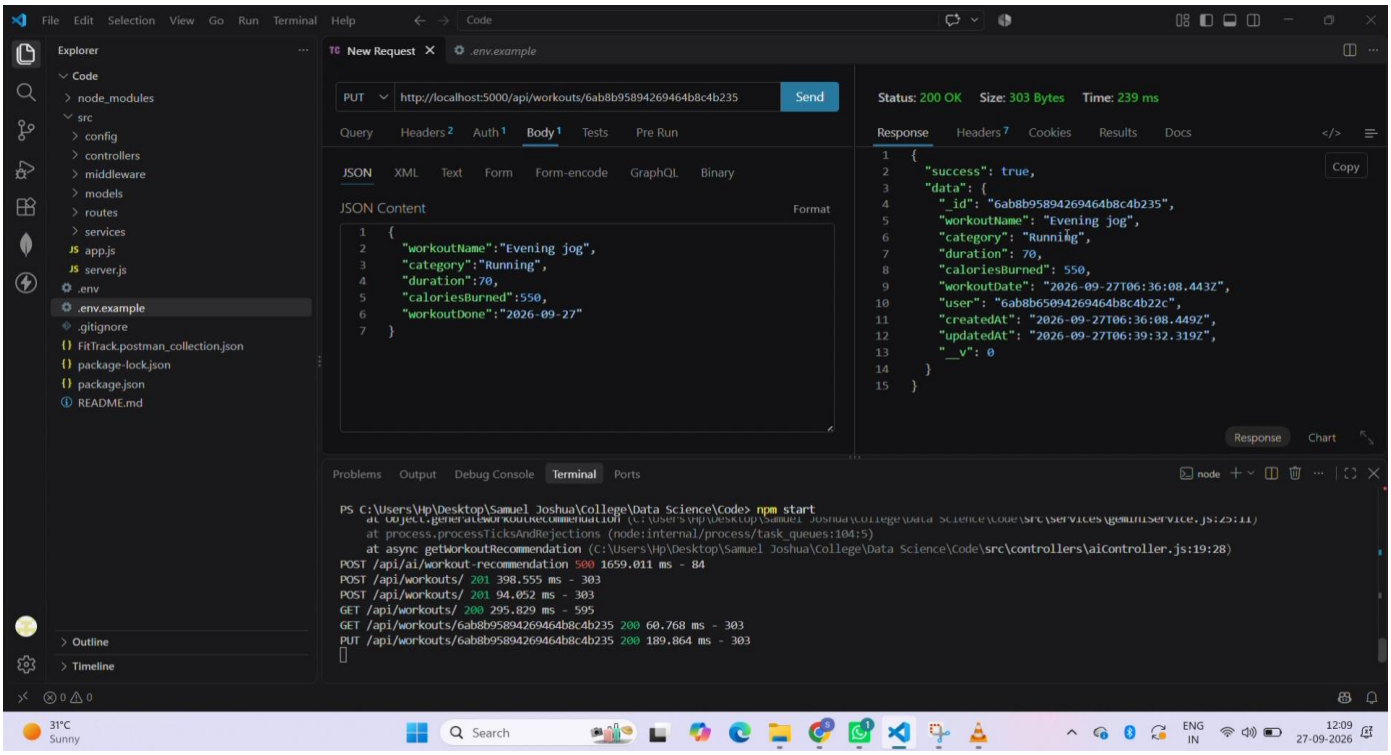
Get Profile



Send the JWT in the Authorization header and verify that the authenticated user's profile information is returned.

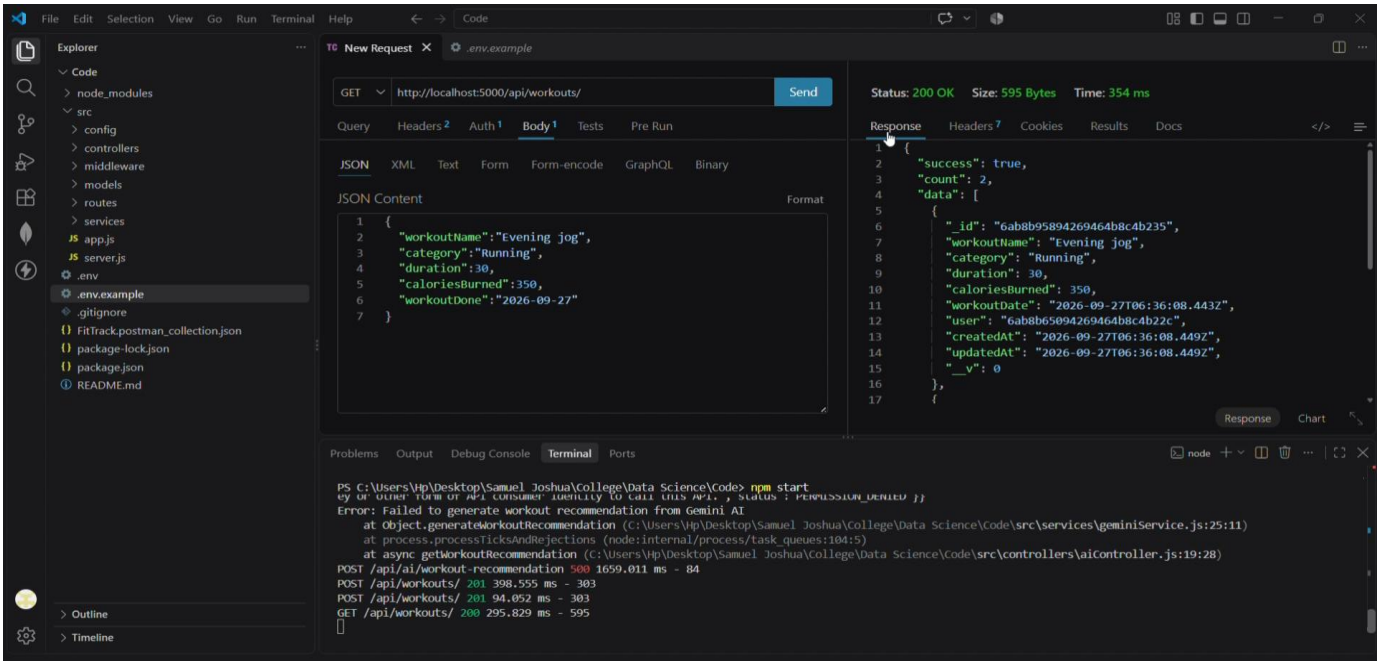
Workout API Testing

Add Workout



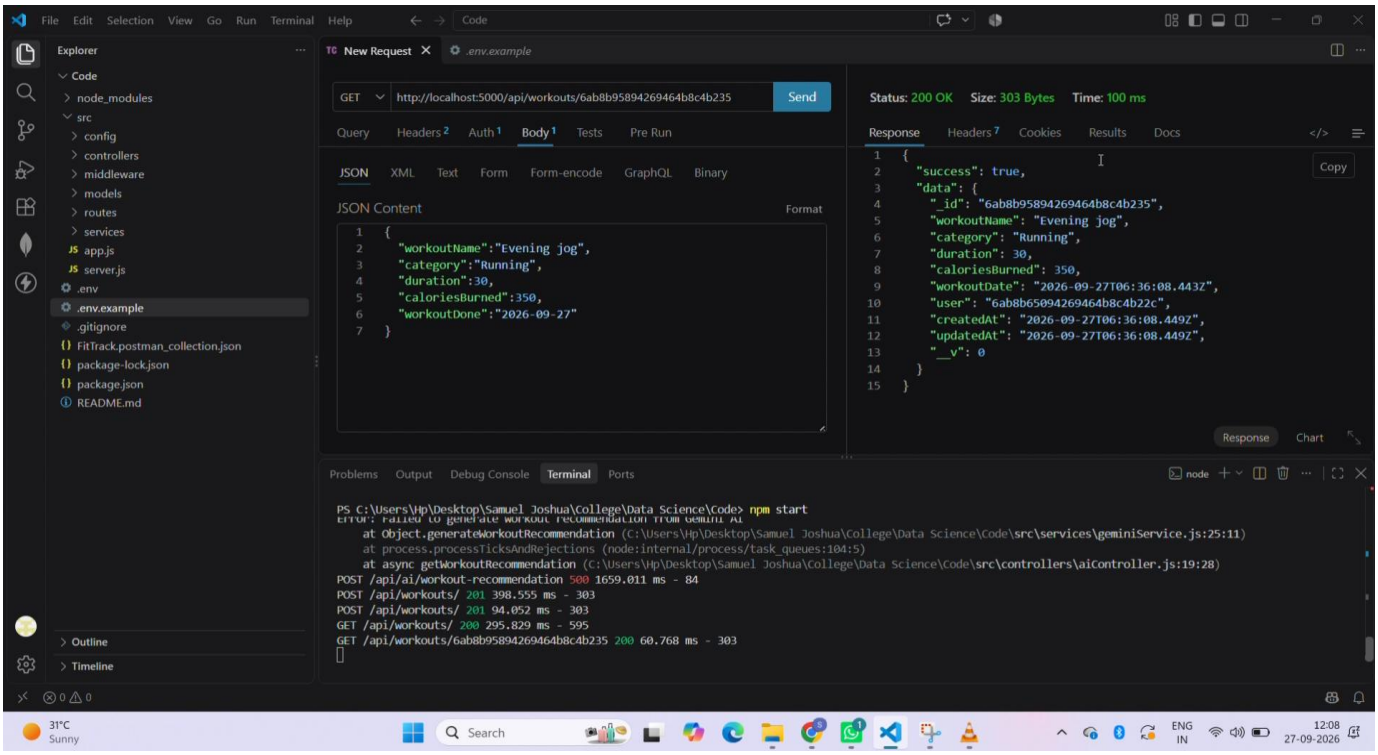
Submit a valid workout request with authentication and verify that the new record is saved for the logged-in user.

Get All Workouts



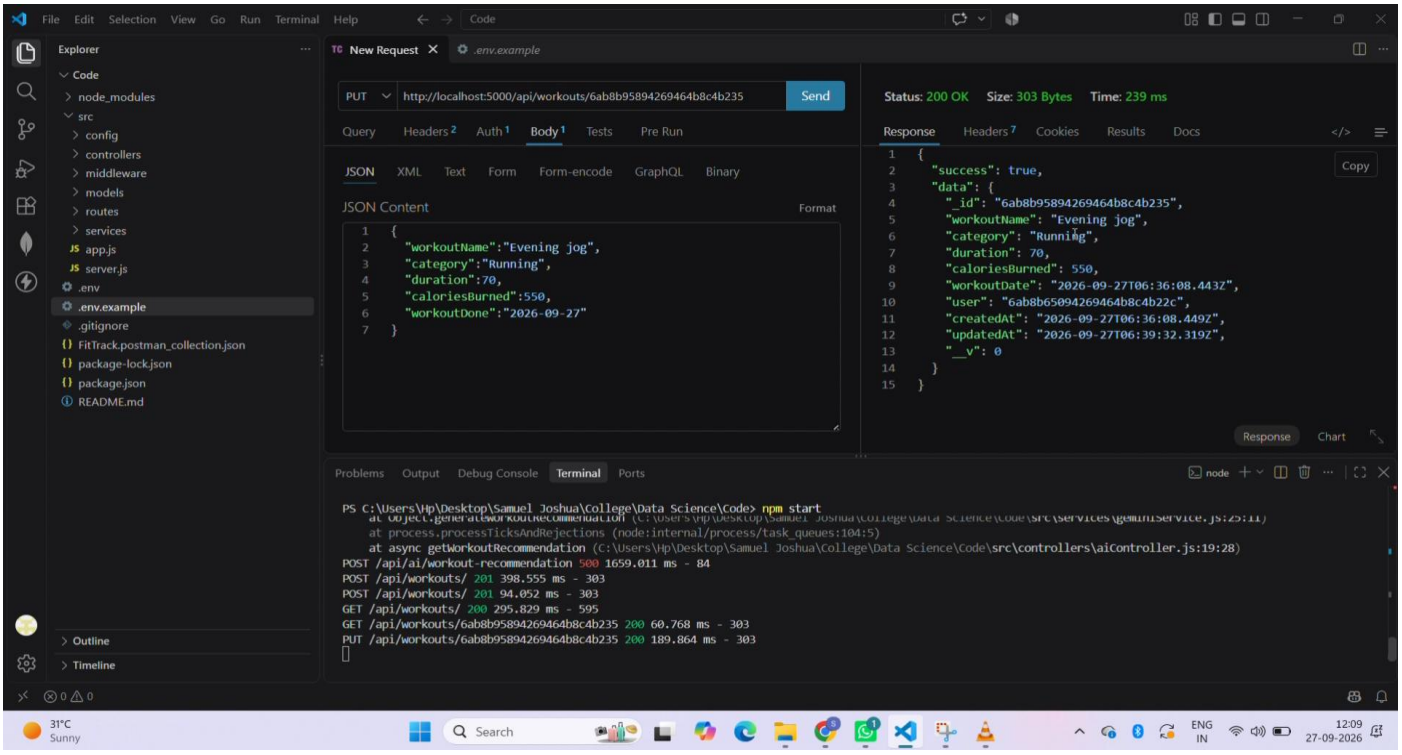
Retrieve the authenticated user's workout history and verify the returned JSON records.

Get Workout By ID



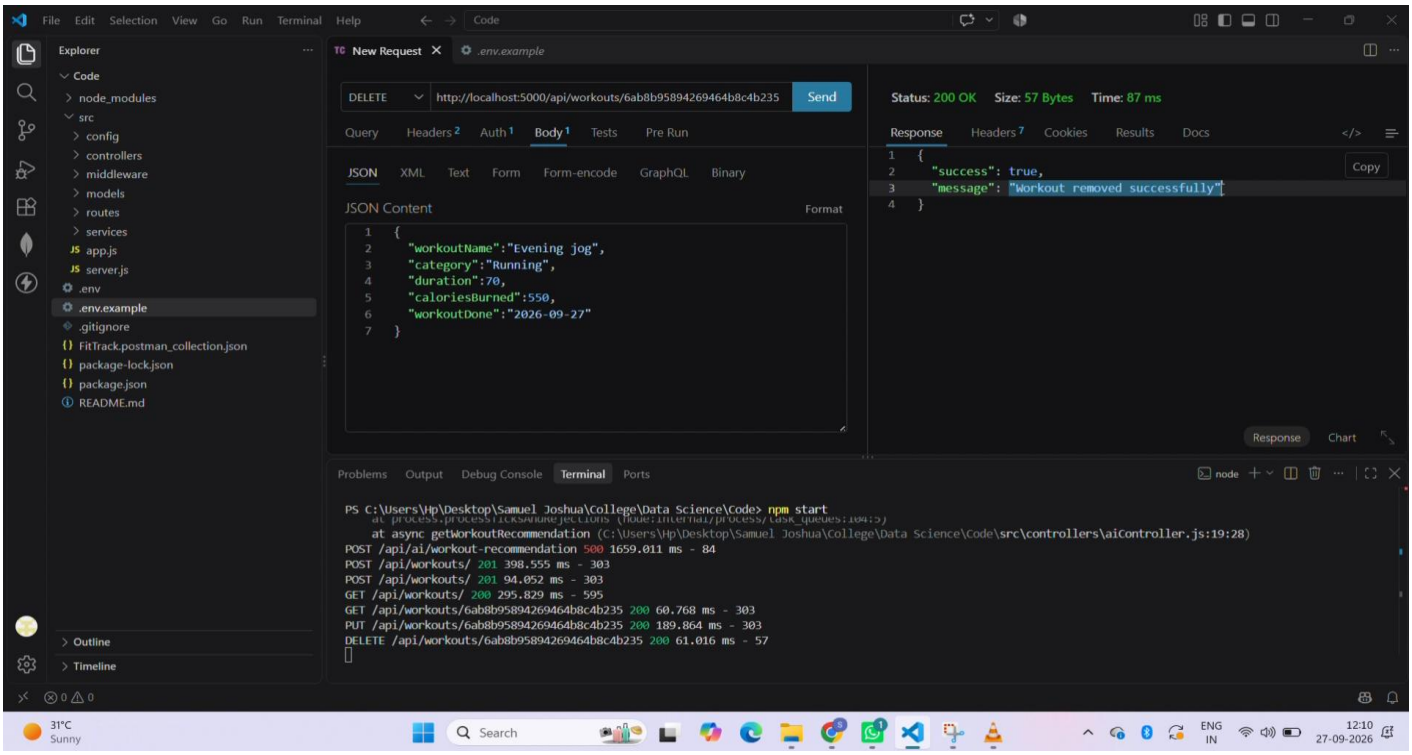
Provide a valid workout ID and verify that the corresponding workout is returned.

Update Workout



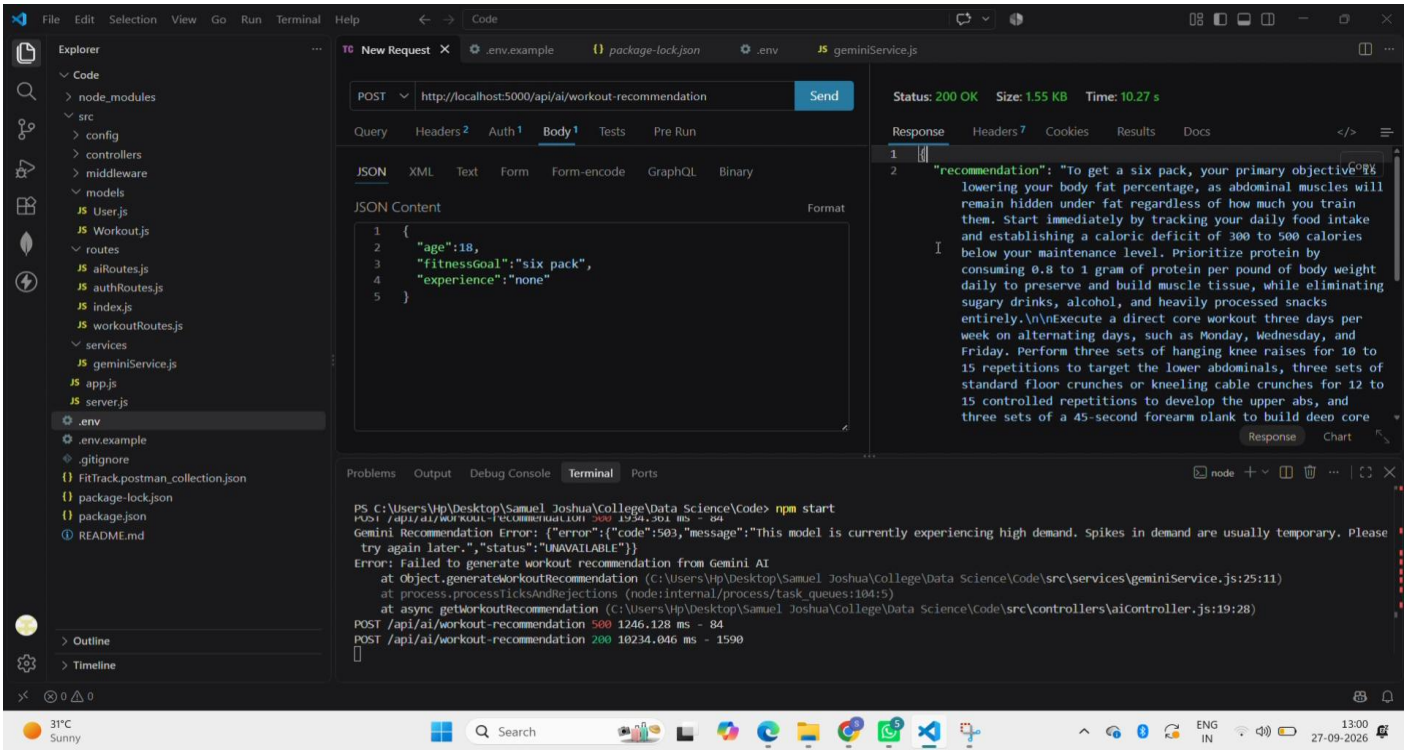
Modify an existing workout and confirm that the updated values are stored correctly.

Delete Workout



Delete a selected workout record and verify the corresponding success response.

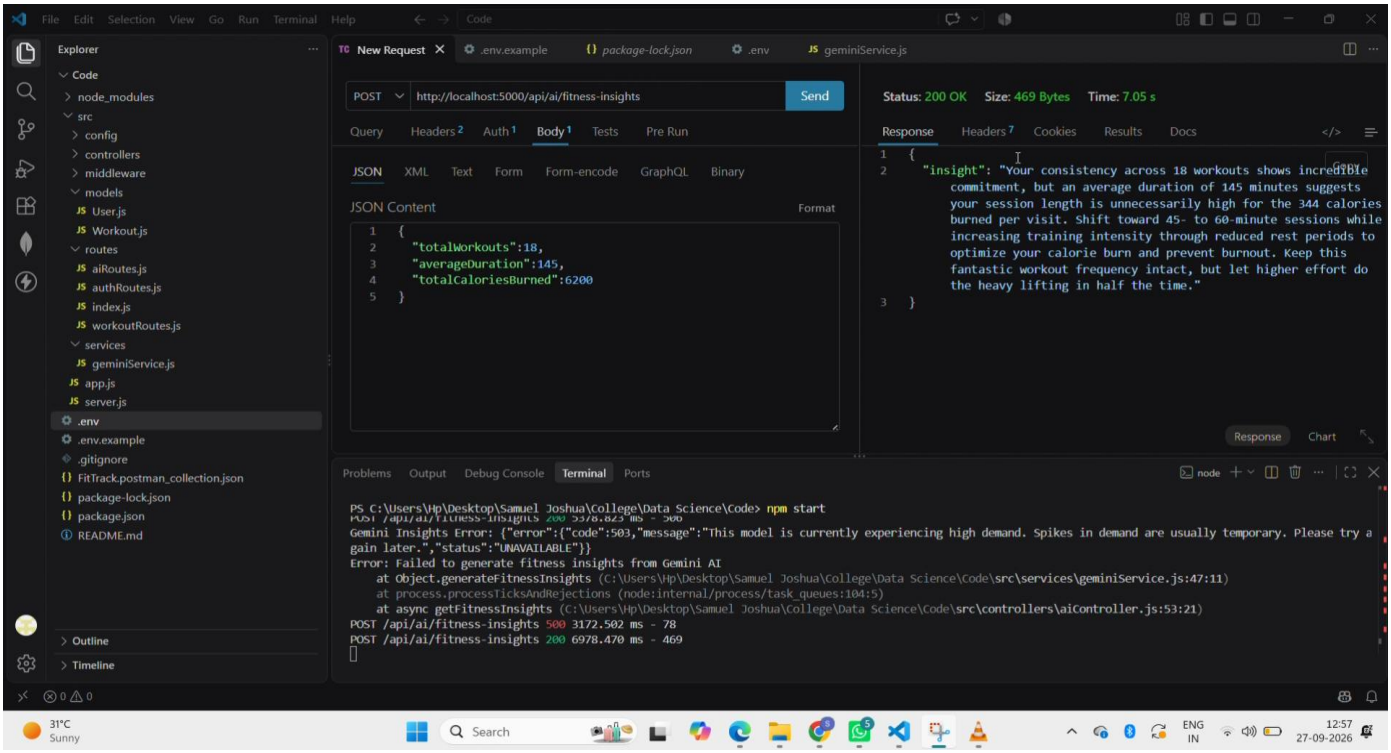
AI API Testing



Workout Recommendation

Submit age, fitness goal, and experience level to the AI endpoint and verify the personalized recommendation returned by Gemini.

Fitness Insights



Submit total workouts, average workout duration, and calories burned and verify that the AI service returns fitness observations and improvement suggestions.

Demo Video Link: https://drive.google.com/file/d/1y3e7yzdV-9jFgJ59REiLeqGZxNba4Iuo/view?usp=sharing